

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Evaluación y Mejora Sistemática de Prompts

Módulo 6 · Ingeniería de Prompts

Golden datasets, LLM-as-judge, DSPy, A/B testing de prompts, versionado y el proceso riguroso de mejora basada en datos.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Evaluación y Mejora Sistemática de Prompts

El prompt engineering sin evaluación es artesanía intuitiva. Con evaluación sistemática, se convierte en ingeniería. La diferencia entre los equipos que producen sistemas LLM robustos y los que producen sistemas frágiles está casi siempre en el rigor de su proceso de evaluación: los primeros tienen golden datasets, métricas definidas, y procesos de mejora iterativa documentados; los segundos prueban manualmente hasta que "parece que funciona" y se sorprenden cuando falla en producción.

Este tema cubre las herramientas y metodologías para evaluar la calidad de los prompts de forma rigurosa y escalarlas al proceso de desarrollo de sistemas LLM. Incluye desde la construcción del golden dataset hasta el uso de evaluación automática con LLM-as-judge, pasando por técnicas de prompt optimization que automatizan la búsqueda del mejor prompt.

Las dimensiones de evaluación de un prompt

Precisión: ¿el output es correcto?

La precisión mide qué fracción de los casos producen el output correcto o aceptable. Para tareas con respuesta discreta (clasificación, extracción, QA con respuesta definida), se puede calcular con un golden dataset comparando automáticamente el output del LLM con la respuesta correcta. Para tareas generativas (resumen, redacción), se necesita evaluación humana o LLM-as-judge para estimar la calidad del output.

Consistencia: ¿el output es reproducible?

La consistencia mide la variabilidad del output con el mismo input. Con temperatura 0, la mayoría de los modelos son completamente deterministas. Con temperatura > 0 , hay variabilidad. Para tareas donde la consistencia es crítica (decisiones reguladas, sistemas de scoring), la consistencia debe medirse explícitamente generando múltiples respuestas para los mismos inputs y calculando la varianza.

Robustez: ¿el output es correcto en casos edge?

La robustez mide el rendimiento en los casos más difíciles, atípicos, o problemáticos. Un prompt puede tener 95% de precisión en los casos comunes pero fallar en todos los casos edge. Construye un edge case dataset específicamente con los tipos de inputs que más probablemente fallarán: entradas muy largas, idiomas distintos del esperado, contenido ambiguo, inputs malformados, intentos de inyección. El rendimiento en el edge case dataset es frecuentemente más informativo que el rendimiento en el dataset general.

Técnicas de evaluación automática

LLM-as-judge para evaluación a escala

LLM-as-judge usa un LLM para evaluar la calidad de los outputs de otro LLM a escala. El juez recibe el input, el output evaluado, y opcionalmente una respuesta de referencia, y produce una puntuación o un juicio cualitativo. Para calibrar el juez: evalúa una muestra de 100 outputs tanto con el juez automático como con evaluadores humanos y verifica que la correlación es suficientemente alta (>0.8) para el tipo de tarea. Un juez bien calibrado permite escalar la evaluación a miles o decenas de miles de casos por coste marginal mínimo.

Prompt optimization automática

DSPy (Stanford, 2023) es un framework que automatiza la optimización de prompts: dado un dataset de ejemplos con inputs y outputs correctos, DSPy optimiza el prompt iterativamente para maximizar la métrica de evaluación. En lugar de iterar manualmente sobre el prompt, el framework explora el espacio de prompts de forma sistemática. Funciona mejor para pipelines complejos de múltiples módulos donde la optimización manual sería prohibitivamente cara.

TextGrad (Yuksekgonul et al., 2025) es otra herramienta de optimización de prompts que usa retroalimentación del LLM como señal de gradiente para actualizar el prompt. La analogía con backpropagation: en lugar de gradientes numéricos, usa descripciones textuales de cómo mejorar cada componente del prompt. Es más intuitivo que DSPy para casos de uso donde el prompt tiene estructura semántica clara.

SECCIÓN 4 · EJEMPLOS REALES

- A/B testing de prompts en producción: un sistema de clasificación de emails tiene dos versiones del prompt en producción simultáneamente. El 10% del tráfico va a la versión B (nueva) y el 90% a la versión A (actual). Se mide la precisión de cada versión sobre los casos etiquetados que llegan en el periodo de prueba. Después de 1.000 casos etiquetados, si la versión B tiene precisión significativamente mayor (test estadístico), se despliega globalmente.
- Golden dataset de extracción de contratos (bufete legal): 200 contratos anotados manualmente por abogados especificando exactamente qué campos deben extraerse y con qué valores. Cada actualización del prompt se evalúa sobre estos 200 contratos antes del despliegue. La tasa de extracción correcta es la métrica de aceptación: mínimo 92% para producción.
- Evaluación con LLM-as-judge para asistente de atención al cliente: se usa GPT-4 como juez para evaluar la calidad de las respuestas del asistente (basado en un modelo más pequeño) en dimensiones de precisión, utilidad, tono y adecuación al cliente. La correlación del juez con evaluadores humanos es 0.87, suficientemente alta para usarlo como métrica de monitorización continua.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Golden dataset demasiado pequeño o no representativo. Un golden dataset de 20 casos evalúa demasiado poco para distinguir diferencias de rendimiento estadísticamente significativas entre prompts. Un mínimo de 50-100 casos para evaluación rápida, 200-500 para evaluación rigurosa de producción.

Error 2: Evaluar solo los casos de éxito. Evalúa también los casos de fallo: ¿en qué porcentaje el modelo falla silenciosamente (produce un output que parece correcto pero no lo es)? Los fallos silenciosos son más peligrosos que los fallos ruidosos. Incluye casos donde la respuesta correcta es "no sé" para evaluar si el modelo admite incertidumbre correctamente.

Error 3: No versionar los prompts. Cada cambio al prompt es una nueva versión del sistema. Sin versionado, es imposible reproducir resultados históricos, hacer rollback a una versión anterior, o entender qué cambio produjo una mejora o degradación. Los prompts deben estar en control de versiones junto con el código.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de mejora de prompts basada en datos: construye el golden dataset (50-200 casos representativos con outputs correctos anotados). Define la métrica de evaluación principal (precisión, F1, calidad LLM-as-judge). Establece el baseline con el prompt actual. Identifica los tipos de fallos más frecuentes (agrupa los errores por categoría). Diseña mejoras del prompt específicas para cada categoría de fallos. Evalúa cada mejora sobre el golden dataset completo. Despliega solo si la mejora es estadísticamente significativa y no hay regresión en otras categorías.

Para versionado de prompts: usa Git para los archivos de configuración de prompts. Cada versión del prompt tiene un hash de commit que permite reproducirla exactamente. Los experimentos se registran con (hash_del_prompt, dataset_de_evaluacion, métricas). Herramientas como Weights & Biases o MLflow pueden trackear experimentos de evaluación de prompts de la misma forma que experimentos de ML.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M5-T7 (Evaluación de LLMs): la evaluación de prompts es la aplicación práctica de los principios de evaluación de LLMs a nivel de sistema específico.
- M8 (LLMOps): el versionado de prompts y la evaluación continua en producción son componentes centrales de LLMOps.
- M3-T6 (Validación cruzada): los principios de separación de datasets de evaluación y train se aplican directamente a la evaluación de prompts.

SECCIÓN 8 · RESUMEN EJECUTIVO

La evaluación sistemática de prompts requiere: golden dataset representativo (50-500 casos con outputs correctos), métricas definidas (precisión, consistencia, robustez en casos edge), proceso iterativo de mejora basado en análisis de fallos, LLM-as-judge para escalar la evaluación a miles de casos, y versionado de prompts en Git. Herramientas de optimización automática: DSPy para pipelines complejos, TextGrad para optimización basada en retroalimentación textual. Los fallos silenciosos son más peligrosos que los ruidosos: evalúa siempre los casos donde la respuesta correcta es admitir incertidumbre. El ROI de la evaluación sistemática se manifiesta en menos fallos en producción y menor tiempo de debugging.